PSD and Wavelet
ECE3204

Power Spectral Estimation using Periodogram

```
clc;
clear;
close all;
fs = 1000;              % Sampling frequency
t = 0:1/fs:1-1/fs;      % Time vector
% Signal with two frequencies
x = sin(2*pi*100*t) + 0.5*sin(2*pi*200*t);
% Power spectral density using periodogram
[pxx,f] = periodogram(x,[],[],fs);
% Plot
figure;
plot(f,10*log10(pxx));
xlabel('Frequency (Hz)');
ylabel('Power/Frequency (dB/Hz)');
title('Power Spectral Density using Periodogram');
grid on;
```

Explanation

- fs → Sampling frequency
- x → Input signal
- periodogram() → Estimates PSD
- pxx → Power spectral density
- f → Frequency vector

Periodogram Formula

The PSD estimate is based on

$$P(f) = \frac{1}{N} |X(f)|^2$$

where

- $X(f)$ = Fourier transform of the signal
- N = number of samples.

Power Spectral Estimation using Welch Method (More Accurate)

```
clc;
clear;
close all;
fs = 1000;
t = 0:1/fs:1-1/fs;
x = sin(2*pi*100*t) + 0.5*sin(2*pi*200*t);
% Welch PSD estimation
[pxx,f] = pwelch(x,[],[],[],fs);
figure;
plot(f,10*log10(pxx));
xlabel('Frequency (Hz)');
ylabel('Power/Frequency (dB/Hz)');
title('Power Spectral Density using Welch Method');
grid on;
```

Welch PSD is an **average of modified periodograms**:

$$P_{welch}(f) = \frac{1}{K}\sum_{k=1}^{K} P_k(f)$$

Where

- K = number of segments
- $P_k(f)$ = periodogram of each segment.

Power Spectral Estimation using Welch Method (More Accurate)

Mathematical Concept

Welch PSD is an **average of modified periodograms**:

$$P_{welch}(f) = \frac{1}{K}\sum_{k=1}^{K} P_k(f)$$

Where

- K = number of segments
- $P_k(f)$ = periodogram of each segment.

Power Spectral Estimation using FFT

```
clc;
clear;
close all;
fs = 1000;
t = 0:1/fs:1-1/fs;
x = sin(2*pi*100*t) + 0.5*sin(2*pi*200*t);
N = length(x);
X = fft(x);
Pxx = (1/(fs*N)) * abs(X).^2;
f = (0:N-1)*(fs/N);
figure;
plot(f,Pxx);
xlabel('Frequency (Hz)');
ylabel('Power');
title('Power Spectral Estimation using FFT');
grid on;
```

Power spectrum is calculated as:

$$P(f) = \frac{|X(f)|^2}{N}$$

Where

- $X(f)$ = FFT of the signal
- N = number of samples.

AR PSD using Yule-Walker Method (pyulear)

This is the simplest AR PSD estimation method.

```
clc;
clear;
close all;
fs = 1000;              % Sampling frequency
t = 0:1/fs:1-1/fs;
% Test signal
x = sin(2*pi*100*t) + 0.5*sin(2*pi*200*t) + 0.2*randn(size(t));
order = 8;              % AR model order
% PSD estimation using Yule-Walker
[pxx,f] = pyulear(x, order, 1024, fs);
figure;
plot(f,10*log10(pxx));
xlabel('Frequency (Hz)');
ylabel('Power/Frequency (dB/Hz)');
title('AR PSD Estimation using Yule-Walker Method');
grid on;
```

Explanation

- x → Input signal
- order → Order of the AR model
- pyulear() → Computes PSD using Yule–Walker equations
- pxx → Power spectral density
- f → Frequency vector

Yule–Walker PSD Formula

The PSD of an AR process is

$$P(f) = \frac{\sigma^2}{\left|1 + \sum_{k=1}^{p} a_k e^{-j2\pi fk}\right|^2}$$

Where

- p = AR model order
- a_k = AR coefficients
- σ^2 = white noise variance.

AR PSD using Burg Method

Burg's method usually gives better frequency resolution.

```
clc;
clear;
close all;
fs = 1000;
t = 0:1/fs:1-1/fs;
x = sin(2*pi*100*t) + 0.5*sin(2*pi*200*t) + 0.2*randn(size(t));
order = 8;
% PSD using Burg method
[pxx,f] = pburg(x, order, 1024, fs);
figure;
plot(f,10*log10(pxx));
xlabel('Frequency (Hz)');
ylabel('Power/Frequency (dB/Hz)');
title('AR PSD Estimation using Burg Method');
grid on;
```

Typical AR Model Orders

Signal Length	Recommended Order
Small data	4 – 10
Medium data	8 – 20
Large data	20+

ARMA (Autoregressive Moving Average)

ARMA (Autoregressive Moving Average) is a statistical model used in signal processing and time-series analysis to represent a signal using both autoregressive (AR) and moving average (MA) components.
It is widely used in spectral estimation and can be implemented in MATLAB.

An **ARMA(p, q)** model combines:

- **AR(p)** → Autoregressive part of order p
- **MA(q)** → Moving average part of order q

The mathematical form is:

$$x(n) = -\sum_{k=1}^{p} a_k x(n-k) + \sum_{k=0}^{q} b_k w(n-k)$$

Where:

- $x(n)$ = output signal
- a_k = AR coefficients
- b_k = MA coefficients
- $w(n)$ = white noise
- p = AR order
- q = MA order.

MATLAB Example: ARMA Model

```
clc;
clear;
close all;
% AR and MA coefficients
A = [1 -0.9];     % AR coefficients
B = [1 0.5];      % MA coefficients
% Generate white noise
n = randn(1,1000);
% Generate ARMA signal
x = filter(B,A,n);
% Plot signal
plot(x)
title('ARMA Generated Signal')
xlabel('Samples')
ylabel('Amplitude')
grid on
```

ARMA Power Spectral Density

```
clc;
clear;
close all;
A = [1 -0.9];     % AR part
B = [1 0.5];      % MA part
[H,f] = freqz(B,A,1024);
PSD = abs(H).^2;
plot(f/pi,10*log10(PSD))
xlabel('Normalized Frequency')
ylabel('Power (dB)')
title('ARMA Power Spectral Density')
grid on
```

Wavelet (Wavelet Transform)

A Wavelet is a small wave-like function used to analyze signals in both time and frequency simultaneously. Unlike the Fourier Transform, which only shows frequency information, wavelets provide time-frequency localization.
Wavelet analysis is widely used in signal processing, image compression, denoising, and biomedical signal analysis.

A signal is decomposed into **scaled and shifted versions of a wavelet function**.

General formula:

$$W(a,b) = \int x(t)\psi^*\left(\frac{t-b}{a}\right)dt$$

Where:

- $x(t)$ = input signal
- $\psi(t)$ = wavelet function
- a = scale parameter
- b = translation parameter

Types of Wavelet Transform

1Continuous Wavelet Transform (CWT)

• Uses continuous scale and shift
• High resolution but computationally heavy.

2 Discrete Wavelet Transform (DWT)

• Uses discrete scales and positions
• Efficient for digital signal processing.

Common wavelets include:
•Haar
•Daubechies
•Symlets
•Coiflets

MATLAB Example: Discrete Wavelet Transform

Using MATLAB:

```
clc;
clear;
close all;
% Sample signal
t = 0:0.001:1;
x = sin(2*pi*50*t) + sin(2*pi*120*t);
% Perform wavelet decomposition
[c,l] = wavedec(x,3,'db4');
% Extract approximation and detail coefficients
A3 = appcoef(c,l,'db4',3);
D3 = detcoef(c,l,3);
% Plot signal
plot(x)
title('Original Signal')
xlabel('Samples')
ylabel('Amplitude')
```

MATLAB Example: Continuous Wavelet Transform

```
clc;
clear;
close all;
t = 0:0.001:1;
x = sin(2*pi*50*t) + sin(2*pi*120*t);
cwt(x,'amor')
title('Continuous Wavelet Transform')
```